Basics Day 5 — Session 5 (Hours 17–20)

Python Programming (Basic) • Tuples, Sets, and Dictionaries

Session 5 Overview

Topics Covered Today

- Hour 17: Tuples + unpacking
- Hour 18: Sets — uniqueness + membership
- Hour 19: Dictionary fundamentals
- Hour 20: Dictionary iteration + counting pattern

Hour 17: Tuples + Unpacking

Learning Outcomes

- Create a tuple and explain why it differs from a list
- Demonstrate immutability by triggering a `TypeError`
- Unpack a tuple's values into named variables in one line
- Choose between tuple and list based on the use case
- Build a coordinate-tracking program using tuples in a list

What Is a Tuple?

Three defining properties

- **Ordered** — items have positions (first, second, third…)
- **Immutable** — cannot be changed after creation
- **Collection** — one variable holds multiple related values

> If a list is a whiteboard you can erase, a tuple is a **laminated card** — readable, passable, but never rewritable.

Tuple Literal Syntax

```
# Two-element tuple
point = (3, 7)
print(point)          # (3, 7)
print(type(point))    # <class 'tuple'>

# Tuples can hold multiple types
date = (2024, 6, 15)
colour = (255, 128, 0)
measurement = (34.5, "celsius")
```

Indexing works exactly like lists

```
point = (3, 7)
print(point[0])   # 3
print(point[1])   # 7
```

The Single-Element Trap

```
# ❌ NOT a tuple — Python reads () as math grouping
not_a_tuple = (42)
print(type(not_a_tuple))   # <class 'int'>

# ✓ The trailing comma is what makes it a tuple
actually_a_tuple = (42,)
print(type(actually_a_tuple))   # <class 'tuple'>
```

> **Rule:** the *comma* makes the tuple, not the parentheses.

Immutability: Why Tuples Cannot Change

```
point = (3, 7)
point[0] = 99   # 🔴 TypeError!
# TypeError: 'tuple' object does not support item assignment
```

What you *can* do with a tuple

```
point = (3, 7)

print(point[0])          # read by index
print(3 in point)        # membership check → True
for value in point:      # iterate
    print(value)
```

> Immutability = **protection** (prevents accidental changes) + **communication** (signals "this data is fixed")

Tuple Unpacking

```
# Instead of point[0] and point[1]:
point = (3, 7)
x, y = point

print(x)   # 3
print(y)   # 7
```

Count must match exactly

```
point = (3, 7)
# x, y, z = point  # 🔴 ValueError: not enough values to unpack
```

Unpacking any tuple

```
date = (2024, 6, 15)
year, month, day = date

print(f"Year: {year}, Month: {month}, Day: {day}")
```

Swap Trick + Ignoring Values

The Pythonic swap (no temp variable needed)

```
a = 10
b = 20
a, b = b, a
print(f"a={a}, b={b}")   # a=20, b=10
```

Ignoring a value with `_`

```
measurement = (34.5, "celsius")
value, _ = measurement   # _ signals "I don't need this"

print(f"Temperature: {value}")
```

Tuple vs List: Decision Guide

Situation	Best choice
Shopping cart (items added/removed)	**List**
An `(x, y)` coordinate	**Tuple**
A date `(year, month, day)`	**Tuple**
A series of test scores	**List**
RGB colour `(255, 128, 0)`	**Tuple**
User messages accumulating	**List**
Function returning two results	**Tuple**

> **Rule of thumb:** things that *accumulate* → list; fixed *records* with known parts → tuple.

Demo: Coordinate Pairs

```python
# Part 1 — Create and read coordinate tuples
origin = (0, 0)
home = (3, 7)
destination = (-2, 5)

# Part 2 — Unpack and use
x, y = home
print(f"Home x-coordinate: {x}")
print(f"Home y-coordinate: {y}")

# Part 3 — Function returning a tuple
def get_midpoint(a, b):
    return ((a[0] + b[0]) / 2, (a[1] + b[1]) / 2)

mid_x, mid_y = get_midpoint(home, destination)
print(f"Mid x={mid_x}, Mid y={mid_y}")

# Part 4 — List of tuples
path = [(0, 0), (1, 3), (4, 2), (6, 8)]
for point in path:
    x, y = point
    print(f"  x={x}, y={y}")
```

Lab: Coordinate Tracker

Time: 14 minutes

Tasks

1. Create a list `sensors` with **five** `(x, y)` **tuples** (use varied positive and negative values)
2. Use a `for` loop + tuple unpacking to print each sensor:

   ```
   Sensor at x=3, y=7
   ```

3. Extract all x values into `x_values` using a loop + `append()`
4. Print the **minimum** and **maximum** x using `min()` / `max()`

Completion Criteria

✓ Five sensor tuples in a list

Common Pitfalls (Hour 17)

- ⚠️ `(42)` is an `int` — single-element tuple needs a trailing comma: `(42,)`
- ⚠️ `x, y, z = (3, 7)` → `ValueError`: variable count must match tuple length
- ⚠️ Trying to reassign `point[0] = 99` → `TypeError`
- ⚠️ Using a list `[3, 7]` when a tuple `(3, 7)` is the clearer choice for a fixed record

Quick Check

Question: A list stores 10 RGB colour values that will never change. Should you use a list of lists or a list of tuples? Why?

Hour 18: Sets — Uniqueness + Membership

Learning Outcomes

- Create a set using a literal and the `set()` constructor
- Add items with `add()` and remove safely with `discard()`
- Check membership with `in` and explain why sets are fast
- Use union (`|`), intersection (`&`), and difference (`-`)
- Build the Unique Visitors program

What Is a Set?

The bag-of-unique-tags analogy

> A set is like a bag where each tag must be unique. If you drop in a tag that already exists, the bag rejects it — silently.

Lists vs Sets at a glance

Feature	List	Set
Ordered	✓	✗
Allows duplicates	✓	✗
Fast membership check	slower	**faster**

Creating Sets

```
# Set literal — curly braces, comma-separated
fruits = {"apple", "banana", "cherry"}
print(fruits)          # order may vary!

# Duplicates are silently discarded
colours = {"red", "blue", "red", "green", "blue"}
print(colours)         # {'red', 'blue', 'green'}
print(len(colours))    # 3

# Convert a list to a set — the duplicate-removal pattern
names_list = ["Alice", "Bob", "Alice", "Carol", "Bob"]
unique_names = set(names_list)
print(unique_names)    # {'Alice', 'Bob', 'Carol'}
print(len(unique_names))  # 3
```

The Empty Set Trap

```
# ❌ This creates an empty DICTIONARY, not a set
wrong = {}
print(type(wrong))   # <class 'dict'>

# ✓ Empty set MUST use set()
correct = set()
print(type(correct))  # <class 'set'>
```

> **Rule:** `{}` = empty dict. `set()` = empty set. Always.

Sets only hold hashable values

```
valid_set = {"hello", 42, (1, 2)}   # ✓ strings, ints, tuples

# invalid_set = {[1, 2, 3]}         # 🔴 TypeError: unhashable type: 'list'
```

Adding and Removing Items

```
visitors = {"Alice", "Bob"}

# add() — duplicate add does nothing
visitors.add("Carol")
visitors.add("Alice")   # already there; no error, no duplicate
print(visitors)         # {'Alice', 'Bob', 'Carol'}

# discard() — safe removal (no error if missing)
visitors.discard("Zara")    # Zara not there; no crash
visitors.discard("Bob")     # Bob is there; removed

# remove() — raises KeyError if item not found
try:
    visitors.remove("Zara")
except KeyError as e:
    print(f"KeyError: {e}")
```

> **Rule of thumb:** when unsure if the item exists, use `discard()`.

Membership Checks

```
approved = {"alice", "bob", "carol", "david"}

# in operator — reads like English
if "carol" in approved:
    print("Welcome, carol!")

if "zara" not in approved:
    print("Access denied.")
```

Why sets are faster than lists for membership

- **List `in`**: scans item-by-item from the start
- **Set `in`**: uses *hashing* — jumps directly to the answer

> **Pattern:** If you check membership many times and don't need order or duplicates → **use a set**.

Set Operations

```python
python_workshop = {"Alice", "Bob", "Carol", "David", "Eva"}
data_workshop   = {"Carol", "Eva", "Frank", "Grace", "David"}

# Union | — everyone in either group
all_students = python_workshop | data_workshop
# {'Alice', 'Bob', 'Carol', 'David', 'Eva', 'Frank', 'Grace'}

# Intersection & — only those in BOTH groups
attended_both = python_workshop & data_workshop
# {'Carol', 'Eva', 'David'}

# Difference - — in first but NOT in second
python_only = python_workshop - data_workshop
# {'Alice', 'Bob'}
```

Operator	Question answered
`|`	Who is in either group?

Sets Are Unordered — Use `sorted()`

```
tags = {"python", "beginner", "coding", "tutorial"}

# ❌ Order is unpredictable — don't rely on it
print(tags)              # {'free', 'beginner', ...} varies

# ✓ Use sorted() for consistent display
print(sorted(tags))      # ['beginner', 'coding', 'python', 'tutorial']

# Loop over a set in sorted order
for tag in sorted(tags):
    print(f"  - {tag}")
```

> `sorted()` returns a **new list** — your set is unchanged.

Demo: List of Names → Set

```python
# Conference sign-in sheet (with duplicates)
sign_ins = [
    "Alice", "Bob", "Alice", "Carol",
    "Bob", "David", "Bob", "Alice", "Eva", "Carol",
]

print(f"Total sign-ins: {len(sign_ins)}")   # 10

# One call removes all duplicates
unique_visitors = set(sign_ins)
print(f"Unique attendees: {len(unique_visitors)}")  # 5

# Display sorted
for name in sorted(unique_visitors):
    print(f"  - {name}")

# Membership check
if "Alice" in unique_visitors:
    print("Alice attended.")
if "Zara" not in unique_visitors:
    print("Zara did NOT attend.")
```

Lab: Unique Visitors

Time: 15 minutes

Tasks

1. Ask the user to enter **10 names** (encourage repeats)
2. Store each name in a list using `append()`
3. Convert the list to a set after all names are collected
4. Print the **unique count** and the **sorted unique names**
5. Add a comment in your code explaining why duplicates disappear

Completion Criteria

✓ Unique count is correct
✓ Output displays sorted, unique names
✓ At least one comment explains the set's uniqueness property

Common Pitfalls (Hour 18)

- ⚠️ `{}` creates a dict — use `set()` for an empty set
- ⚠️ Iterating a set without `sorted()` gives unpredictable output order
- ⚠️ Trying to put a list inside a set → `TypeError: unhashable type`
- ⚠️ Modifying a set while looping over it → `RuntimeError`

Quick Check

Question: You have a list of 50,000 email addresses and need to check thousands of times whether a given address is in it. Should you keep it as a list or convert to a set? Why?

Hour 19: Dictionary Fundamentals

Learning Outcomes

- Create a dictionary with literal syntax and `dict()`
- Access values by key and explain `KeyError`
- Add new entries and update existing ones
- Use `get(key)` and `get(key, default)` for safe access
- Build an interactive Inventory Manager without crashing on missing keys

What Is a Dictionary?

The real-world analogy

> Like a physical dictionary: the **word** is the key, the **definition** is the value. You jump straight to the entry — no scanning every page.

Key → Value mental model

```
"apples"   →   12
"bananas"  →    5
"oranges"  →    8
"pears"    →    3
"grapes"   →   20
```

> Two parallel lists break when one is sorted or re-ordered. A dictionary keeps each pair **locked together**.

Creating a Dictionary

```
# Dictionary literal: {key: value, ...}
inventory: dict[str, int] = {
    "apples": 12,
    "bananas": 5,
    "oranges": 8,
    "pears": 3,
    "grapes": 20,
}

print(inventory)
print(len(inventory))   # 5

# dict() constructor — keys must be valid identifiers
scores = dict(alice=95, bob=87, carol=91)

# Empty dictionary — {} is safe here (not a set!)
empty: dict[str, int] = {}
```

Reading Values by Key

```
inventory = {"apples": 12, "bananas": 5, "oranges": 8}

# Bracket notation — fast, direct
print(inventory["apples"])    # 12
print(inventory["bananas"])   # 5

# 🔴 KeyError if key is missing
print(inventory["mangoes"])
# KeyError: 'mangoes'
```

> A `KeyError` means: "I searched the whole dictionary and found no key called that." The program stops. We fix this with `get()` in the next slide.

Adding and Updating Entries

```
inventory = {"apples": 12, "bananas": 5}

# Add a new key — same syntax as update
inventory["mangoes"] = 7
print(len(inventory))          # 3

# Update an existing key
inventory["apples"] = 22       # overwrites 12
print(inventory["apples"])     # 22

# Compound assignment — very common for counters
inventory["apples"] += 10      # 22 + 10 = 32

# in operator checks KEYS
print("apples" in inventory)   # True
print("durians" in inventory)  # False
```

> ⚠️ A typo in a key *silently creates a new entry* — no error raised.

Safe Access with `get()`

```
inventory = {"apples": 12, "bananas": 5}

# get(key) → returns None if key missing (no crash)
result = inventory.get("mangoes")
print(result)           # None

# get(key, default) → returns your fallback value
count = inventory.get("mangoes", 0)
print(count)            # 0

count = inventory.get("apples", 0)
print(count)            # 12  (default ignored when key exists)
```

Three patterns side by side

```
# 1. Bracket — use when key is CERTAIN to exist
qty = inventory["apples"]
```

Removing Entries

```python
inventory = {"apples": 12, "bananas": 5, "pears": 3}

# del — direct removal (KeyError if missing)
del inventory["pears"]

# pop() — remove AND return the value
removed = inventory.pop("bananas")
print(f"Removed bananas, quantity was: {removed}")

# pop() with a default — safe even if key is missing
missing = inventory.pop("durians", 0)
print(f"Removed durians: {missing}")   # 0, no crash

# clear() — empty the entire dictionary
temp = {"a": 1, "b": 2}
temp.clear()
print(temp)   # {}
```

Keys Must Be Unique and Hashable

```
# Duplicate key in a literal — LAST value wins silently
prices = {
    "apple": 1.20,
    "banana": 0.50,
    "apple": 2.50,   # ⚠️ duplicate!
}
print(prices)   # {'apple': 2.50, 'banana': 0.50} — 1.20 is gone

# Valid key types: str, int, float, bool, tuple (if all elements hashable)
d = {"name": "Alice", 42: "answer", (1, 2): "coord"}

# ❌ Lists cannot be keys — mutable = unhashable
# bad = {[1, 2]: "value"}   # TypeError: unhashable type: 'list'
```

> For Basics: use **string or integer keys** — always safe and clear.

Demo: Inventory Dictionary

```
# Step 1 — Create
inventory = {"apples": 12, "bananas": 5, "oranges": 8, "pears": 3}

# Step 2 — Crash intentionally (then fix)
# print(inventory["mangoes"])   # 🔴 KeyError

# Step 3 — Fix with get()
count = inventory.get("mangoes", 0)
print(f"Mangoes: {count}")           # 0

# Step 4 — Update + add
inventory["apples"] += 10            # delivery arrived
inventory["mangoes"] = 7             # new item

# Step 5 — Smart bulk update
new_items = {"bananas": 8, "kiwis": 15, "oranges": 4}
for item, qty in new_items.items():
    if item in inventory:
        inventory[item] += qty
    else:
        inventory[item] = qty

for item, qty in inventory.items():
    print(f"  {item:<12} {qty:>4}")
```

Lab: Inventory Manager

Time: 15 minutes

Tasks

1. Create `inventory` with **5 items** and quantities of your choice
2. Print it, labelled `"=== Current Inventory ==="`
3. Ask the user for an item name and quantity to add
4. If item exists → increase its quantity; if not → print a warning (do **not** crash or create a new key)
5. Print the final inventory, labelled `"=== Updated Inventory ==="`

Completion Criteria

✓ Correct key is updated
✓ No crash on a missing key (`get()` or `in`-check used)

Common Pitfalls (Hour 19)

- ⚠️ Bare bracket `d["key"]` on user-supplied key → `KeyError` if missing
- ⚠️ `'Apples'` ≠ `'apples'` — keys are case-sensitive; use `.lower()`
- ⚠️ Typo in key name silently creates a ghost entry
- ⚠️ Duplicate key in literal silently overwrites the earlier value
- ⚠️ `{}` is an empty dict — remember `set()` for an empty set

Quick Check

Question: What is the difference between `inventory["apples"]` and `inventory.get("apples", 0)` when `"apples"` is not a key?

Hour 20: Dictionary Iteration + Counting Pattern

Learning Outcomes

- Iterate over keys, values, and key-value pairs using `keys()`, `values()`, `items()`
- Explain what `dict.items()` returns and why tuple unpacking makes it readable
- Write the frequency-counting loop using `d.get(word, 0) + 1`
- Sort and display results with `sorted(d.items())` and aligned f-strings
- Build a complete Word Counter program

The Three Iteration Views

```python
inventory = {"apples": 12, "bananas": 5, "oranges": 8}

# .keys() — just the keys
for item in inventory.keys():
    print(item)   # apples, bananas, oranges

# .values() — just the values
total = 0
for qty in inventory.values():
    total += qty
print(f"Total stock: {total}")   # 25

# .items() — key-value pairs (the workhorse)
for item, qty in inventory.items():
    print(f"{item}: {qty}")
```

Choosing the Right Iteration Method

```python
inventory = {"apples": 12, "bananas": 5, "oranges": 8}

# Use .keys() — when you only need the keys
print("Available items:")
for item in inventory.keys():
    print(f"  - {item}")

# Use .values() — when you only need the values
total = sum(inventory.values())
print(f"Total stock: {total}")

# Use .items() — when you need BOTH (most common)
print("\nFull inventory:")
for item, qty in inventory.items():
    print(f"  {item:<12} {qty:>4}")
```

> `dict.items()` returns a view of `(key, value)` tuples — unpacking in the loop header is cleaner than `pair[0]` / `pair[1]`.

The Counting Pattern — Step 1

The problem

Count how many times each word appears in a sentence. You don't know which words will appear in advance.

Two cases per word

1. **First time** seeing a word → add it with count `1`
2. **Seen before** → increment its count by `1`

Explicit if-branch approach (clear, verbose)

```
sentence = "the quick brown fox jumps over the lazy dog and the fox"
words = sentence.split()
counts = {}

for word in words:
```

The Counting Pattern — Step 2: `get()` One-Liner

How `get(word, 0)` collapses both cases

```
First time:   counts.get('the', 0) → 0   →  0 + 1 = 1  ✓
Second time:  counts.get('the', 0) → 1   →  1 + 1 = 2  ✓
Third time:   counts.get('the', 0) → 2   →  2 + 1 = 3  ✓
```

The canonical counting loop

```python
sentence = "the quick brown fox jumps over the lazy dog and the fox"
words = sentence.split()
counts: dict[str, int] = {}

for word in words:
    counts[word] = counts.get(word, 0) + 1

print(counts)
```

> Read aloud: *"Set `counts[word]` to whatever is currently stored — defaulting to*

Sorting and Displaying Results

```
# Sort alphabetically by key
for word, count in sorted(counts.items()):
    print(f"{word}: {count}")

# Sort by frequency — highest first
for word, count in sorted(counts.items(),
                          key=lambda pair: pair[1],
                          reverse=True):
    print(f"{word}: {count}")
```

Aligned column output with f-string format specifiers

```
print(f"{'Word':<15} {'Count':>5}")
print("-" * 22)
for word, count in sorted(counts.items(),
                           key=lambda pair: pair[1], reverse=True):
    print(f"{word:<15} {count:>5}")
```

Finding the Most Common Item

```python
counts = {"the": 3, "fox": 2, "quick": 1, "brown": 1}

# max() with key=counts.get — finds key with highest value
most_common = max(counts, key=counts.get)
print(f"Most common: '{most_common}' ({counts[most_common]} times)")
# Most common: 'the' (3 times)

# Guard against empty dictionary
if counts:
    most_common = max(counts, key=counts.get)
    print(f"Most common: '{most_common}'")
```

> An empty dictionary is **falsy** — `if counts:` is `True` only when at least one entry exists.

Demo: Word Frequency Counter

```python
# Step 1: Get input
sentence = input("Enter a sentence: ")
words = sentence.split()

# Step 2: Build frequency dictionary
counts: dict[str, int] = {}
for word in words:
    counts[word] = counts.get(word, 0) + 1

# Step 3: Display results
print(f"\nTotal words: {len(words)} | Unique words: {len(counts)}")
print(f"\n{'Word':<15} {'Count':>5}")
print("-" * 22)
for word, count in sorted(counts.items(),
                          key=lambda pair: pair[1], reverse=True):
    print(f"{word:<15} {count:>5}")

# Step 4: Most common
if counts:
    top = max(counts, key=counts.get)
    print(f"\nMost common: '{top}' ({counts[top]} times)")
```

Lab: Word Counter

Time: 10–12 minutes

Minimum Requirements

1. Prompt the user to enter a sentence
2. Split it into individual words with `.split()`
3. Build a frequency dictionary using `d.get(word, 0) + 1`
4. Print each word and its count using `.items()`

Optional Extensions (finish early?)

- Normalize words to lowercase: `word = word.lower()`
- Strip punctuation with `.replace()` chains
- Sort output alphabetically or by frequency
- Print the most common word

Common Pitfalls (Hour 20)

⚠️ `counts[word] += 1` on a word not yet in the dict → `KeyError` — use `get()` instead

⚠️ Printing `counts` directly instead of iterating with `.items()` — messy output

⚠️ Forgetting `sorted()` — output order is insertion order, not frequency order

⚠️ Case sensitivity: `'Hello'` ≠ `'hello'` — apply `.lower()` before counting

⚠️ Punctuation attached to words: `'fox,'` ≠ `'fox'` — strip with `.replace()`

Quick Check

Question: What does `counts.get('elephant', 0)` return if `'elephant'` was never added to `counts`? Does the call create an entry for `'elephant'`?

Session 5 Wrap-Up

What We Covered Today

Hour 17: Tuples

- Ordered, immutable records — parentheses + commas
- Unpacking into named variables; swap trick; `_` for ignored values
- List of tuples as a natural pattern for fixed records

Hour 18: Sets

- Unordered, unique collections — `set()`, `add()`, `discard()`
- `in` for fast membership checks; `sorted()` for ordered display
- Union `|`, intersection `&`, difference `-`

Hour 19: Dictionary, Frozenset

The Four Collection Types — Summary

Type	Ordered	Mutable	Unique values	Best for
`list`	✓	✓	✗	Sequences that grow/change
`tuple`	✓	✗	✗	Fixed records, function returns
`set`	✗	✓	✓	Deduplication, fast membership
`dict`	✓*	✓	keys only	Key-value lookups, counting

> *Dicts maintain insertion order since Python 3.7.

Scope Guardrail Reminder

Covered today (Basics scope ✓)

✓ All four core collection types

✓ `get()` for safe dict access

✓ `d.get(word, 0) + 1` counting pattern

✓ `sorted()` for ordered display

Not yet (Advanced topics ✗)

✗ `collections.namedtuple` / `Counter` / `defaultdict`

✗ Dict/set comprehensions

✗ Frozensets

✗ Named tuple advanced destructuring

Homework / Practice

Recommended Exercises

1. Build a **grade tracker**: dictionary of student → score; print sorted by score
2. Write a **de-duplication tool**: read a list of words, print unique ones sorted
3. Extend the Word Counter to read from a multi-line string
4. Track **(x, y, z)** sensor positions (3-element tuples) and find extremes

Next Session Preview

Session 6 (Hours 21–24)

- Hour 21: Functions — defining, parameters, return values
- Hour 22: Function scope and default arguments
- Hour 23: Modules — `import`, standard library basics
- Hour 24: Checkpoint 3 (mini-assessment + review)

Questions?

Remember:

- Tuples for fixed records, lists for growing collections
- `set()` (not `{}`) for an empty set
- Always use `get()` when the key might be missing
- `sorted()` before displaying any set or unsorted dict

Thank You!

See you in Session 6!